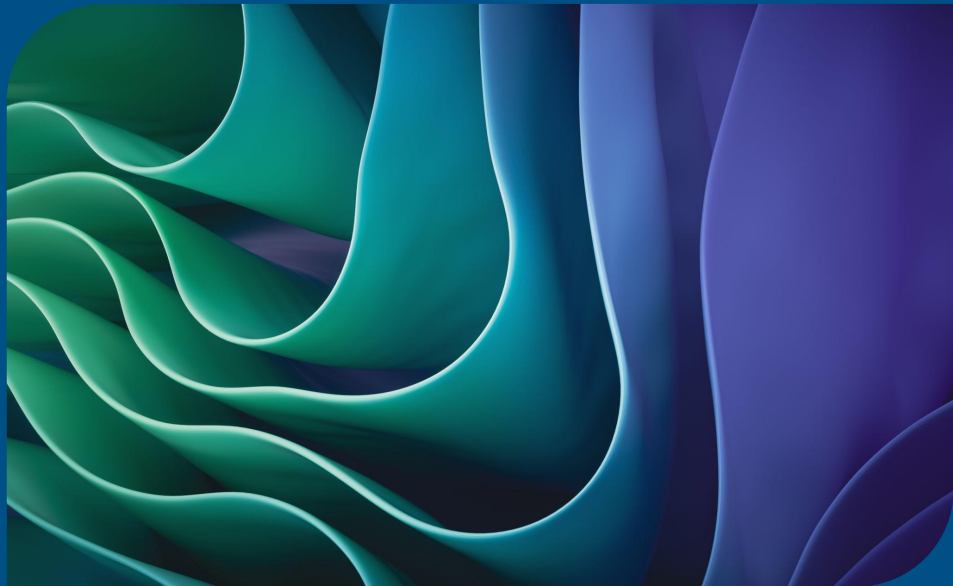


Introduction to Deep Learning

CSCA 5642



IMDB Binary Text Classification

Project Description

This project uses the IMDb Large Movie Review dataset, which contains 25,000 labelled movie reviews for training and 25,000 labelled movie reviews for testing. It's a binary sentiment classification problem where the goal is to classify the IMDb reviews as positive or negative.

Main Goal of the project:

- The main objective of the project is to build and evaluate deep learning models to classify whether the IMDb review is positive or negative.
- I have used the following deep learning models to classify the sentiment of a review.
- I have chosen the best model based on the accuracy, loss values, and F1 score, and implemented the best model on the test set. Hyperparameter tuning(using Keras Tuner) was performed for BiLSTM, CNN, and GRU models.

Citation

Citation:

Data Source Link: <https://ai.stanford.edu/~amaas/data/sentiment/>

Data Source for this project can be found in the below link:

- Sentiment Analysis. (n.d.). Ai.stanford.edu. <https://ai.stanford.edu/~amaas/data/sentiment/>
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011, June 1). Learning Word Vectors for Sentiment Analysis. Association for Computational Linguistics. https://ai.stanford.edu/~amaas/papers/wvSent_acl2011.bib
- @InProceedings{maas-EtAl:2011:ACL-HLT2011, author = {Maas, Andrew L. and Daly, Raymond E. and Pham, Peter T. and Huang, Dan and Ng, Andrew Y. and Potts, Christopher}, title = {Learning Word Vectors for Sentiment Analysis}, booktitle = {Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies}, month = {June}, year = {2011}, address = {Portland, Oregon, USA}, publisher = {Association for Computational Linguistics}, pages = {142--150}, url = {http://www.aclweb.org/anthology/P11-1015} }

Project Steps

Project Description and Objective

Citation

Data Loading

Exploratory Data Analysis

Data Cleaning and Preprocessing

Model Building & Interpretation

Model Summary

Conclusion

Improvements

Challenges

References

Data Loading

```
raw_train_ds = keras.utils.text_dataset_from_directory("aclImdb/train",batch_size=32,validation_split=0.2,subset='training',seed=42)

raw_val_ds = keras.utils.text_dataset_from_directory("aclImdb/train",batch_size=32,validation_split=0.2,subset='validation',seed=42)

raw_test_ds = keras.utils.text_dataset_from_directory("aclImdb/test",batch_size=32)

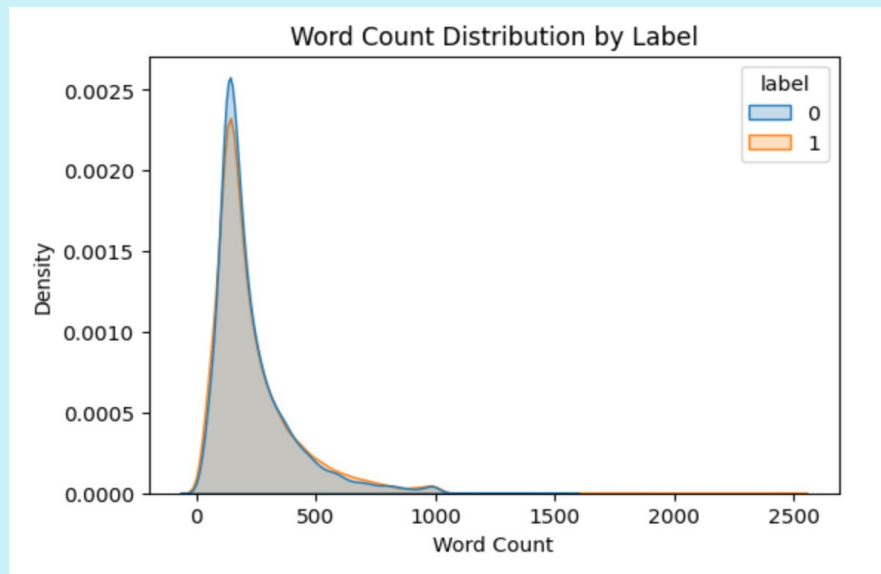
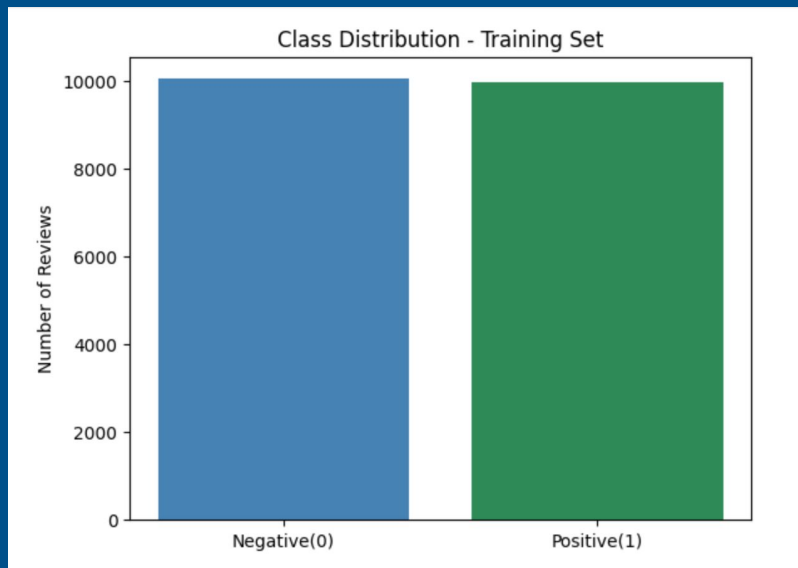
print("\n")
print(f"Number of batches in raw_train_ds:{raw_train_ds.cardinality()}")
print(f"Number of batches in raw_val_ds:{raw_val_ds.cardinality()}")
print(f"Number of batches in raw_test_ds:{raw_test_ds.cardinality()}")
```

```
... Found 25000 files belonging to 2 classes.
Using 20000 files for training.
Found 25000 files belonging to 2 classes.
Using 5000 files for validation.
Found 25000 files belonging to 2 classes.
```

```
Number of batches in raw_train_ds:625
Number of batches in raw_val_ds:157
Number of batches in raw_test_ds:782
```

Exploratory Data Analysis

Class Imbalance & Word Count Distribution

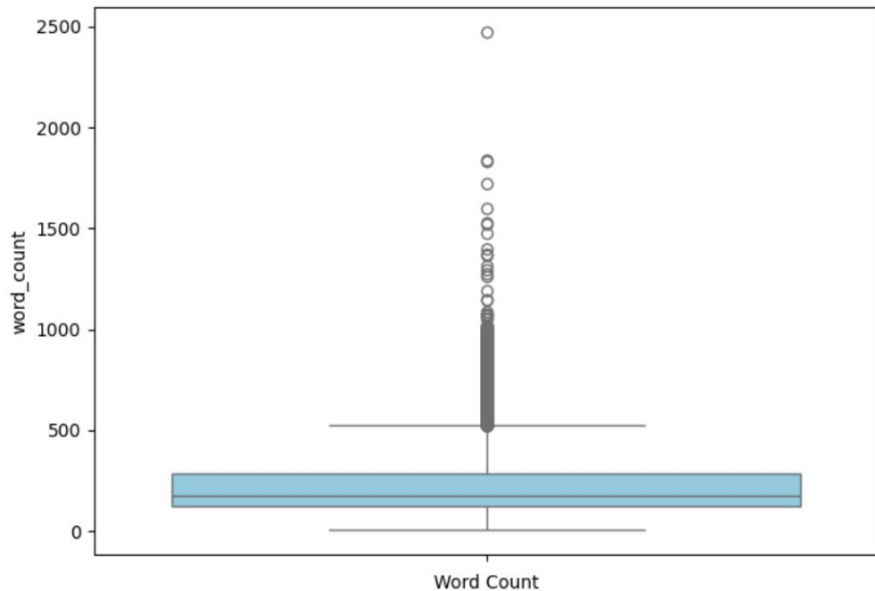


Exploratory Data Analysis

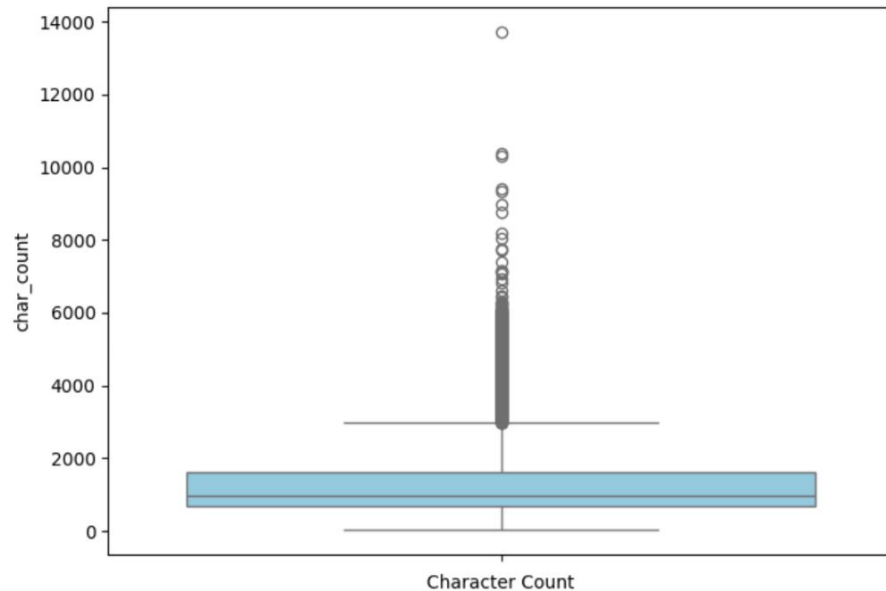
Outliers

<Figure size 1200x500 with 0 Axes>

Word Count Outliers



Character Count Outliers

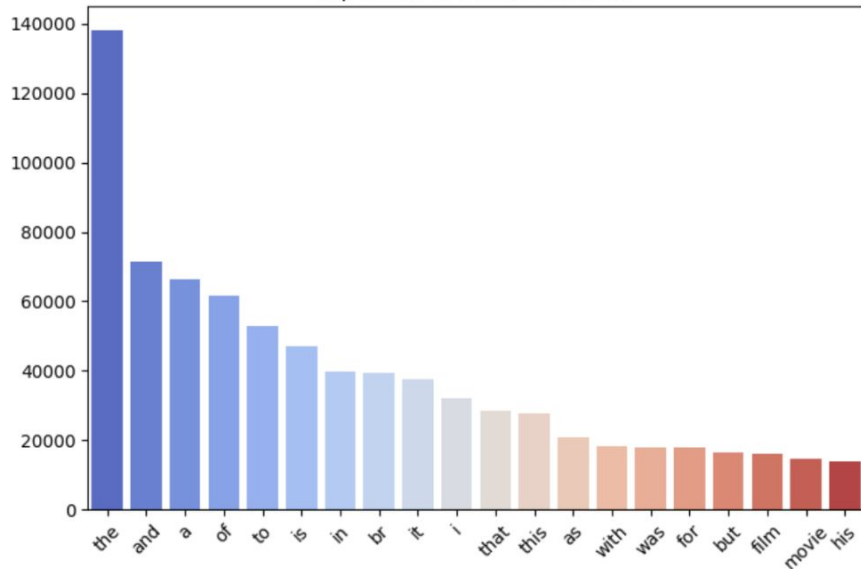


Exploratory Data Analysis

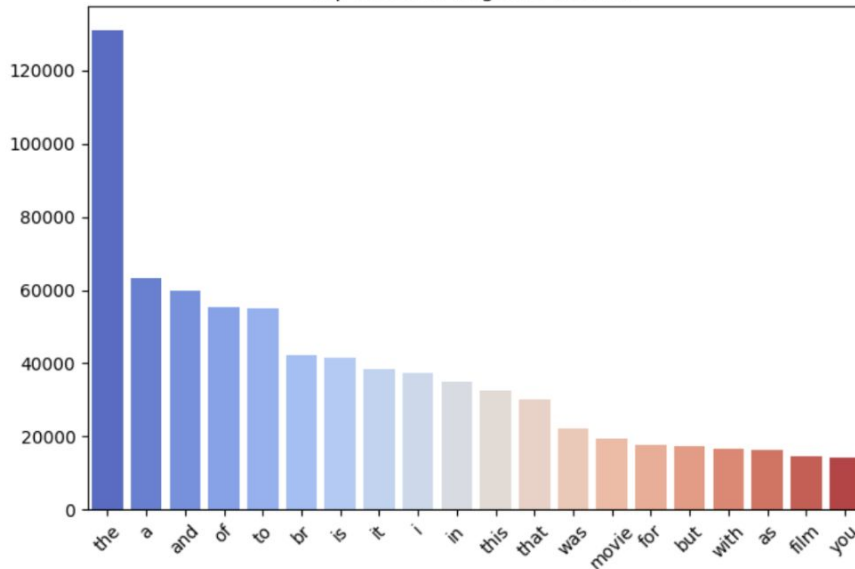
Vocabulary Level EDA

```
/tmp/ipython-input-2436678500.py:25: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
  ax.set_xticklabels(ax.get_xticklabels(),rotation=45)
```

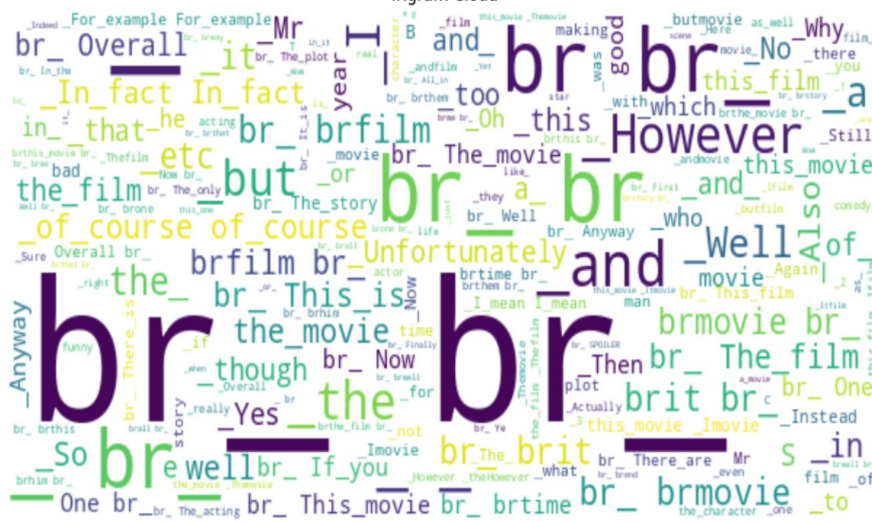
Top Words in Positive Reviews



Top Words in Negative Reviews



Word Cloud & N-grams



N-grams for Positive and Negative Reviews



Data Preprocessing & Cleaning

- Converts the text to lowercase, removes HTML tags, and removes punctuation using a regex pattern
- Text vectorization layer is created to convert the raw text into a sequence of integers.
- Extracts only the text from the training dataset.adapt() learns only the vocabulary from the training data.
- Vectorization is applied to the train, test, and val datasets.
- Optimizing the data pipeline (For faster training - prefetching the next batch when the model trains the current batch)

Model Building & Interpretation

Baseline Model

```
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument 'input_length' is deprecated.
  warnings.warn(
625/625 ————— 5s 4ms/step - accuracy: 0.6023 - loss: 0.6426 - val_accuracy: 0.8526 - val_loss: 0.3704
Epoch 2/5
625/625 ————— 2s 3ms/step - accuracy: 0.8550 - loss: 0.3459 - val_accuracy: 0.8680 - val_loss: 0.3136
Epoch 3/5
625/625 ————— 2s 3ms/step - accuracy: 0.8918 - loss: 0.2675 - val_accuracy: 0.8654 - val_loss: 0.3083
Epoch 4/5
625/625 ————— 2s 4ms/step - accuracy: 0.9096 - loss: 0.2280 - val_accuracy: 0.8640 - val_loss: 0.3179
Epoch 5/5
625/625 ————— 2s 3ms/step - accuracy: 0.9219 - loss: 0.2006 - val_accuracy: 0.8606 - val_loss: 0.3361
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
embedding_4 (Embedding)	(None, 250, 64)	1,280,000
global_average_pooling1d (GlobalAveragePooling1D)	(None, 64)	0
dense_8 (Dense)	(None, 16)	1,040
dense_9 (Dense)	(None, 1)	17

Total params: 3,843,173 (14.66 MB)
Trainable params: 1,281,057 (4.89 MB)
Non-trainable params: 0 (0.00 B)

```
[[2204 258]
 [ 439 2099]]
```

	precision	recall	f1-score	support
0	0.83	0.90	0.86	2462
1	0.89	0.83	0.86	2538
accuracy			0.86	5000
macro avg	0.86	0.86	0.86	5000
weighted avg	0.86	0.86	0.86	5000

Model Building and Interpretation

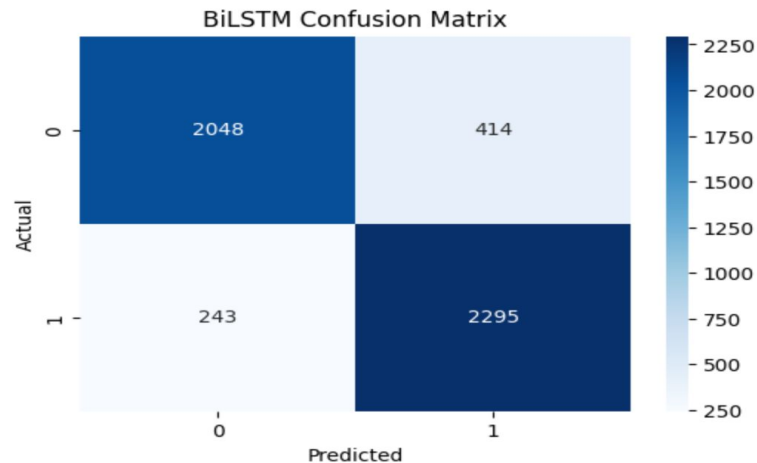
BiLSTM (Standard)

Epoch 1/10
625/625 — 20s 26ms/step - accuracy: 0.6024 - loss: 0.6452 - val_accuracy: 0.7336 - val_loss: 0.5649
Epoch 2/10
625/625 — 17s 27ms/step - accuracy: 0.8065 - loss: 0.4570 - val_accuracy: 0.8388 - val_loss: 0.3846
Epoch 3/10
625/625 — 14s 23ms/step - accuracy: 0.8775 - loss: 0.3228 - val_accuracy: 0.8686 - val_loss: 0.3229
Epoch 4/10
625/625 — 14s 23ms/step - accuracy: 0.9041 - loss: 0.2652 - val_accuracy: 0.8646 - val_loss: 0.3300
Epoch 5/10
625/625 — 15s 24ms/step - accuracy: 0.9142 - loss: 0.2390 - val_accuracy: 0.8672 - val_loss: 0.3688
Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 250, 64)	1,280,000
spatial_dropout1d_1 (SpatialDropout1D)	(None, 250, 64)	0
bidirectional_2 (Bidirectional)	(None, 64)	24,832
dense_6 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_7 (Dense)	(None, 1)	33

Total params: 3,920,837 (14.96 MB)
Trainable params: 1,306,945 (4.99 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2,613,892 (9.97 MB)

	precision	recall	f1-score	support
0	0.89	0.83	0.86	2462
1	0.85	0.90	0.87	2538
accuracy			0.87	5000
macro avg	0.87	0.87	0.87	5000
weighted avg	0.87	0.87	0.87	5000



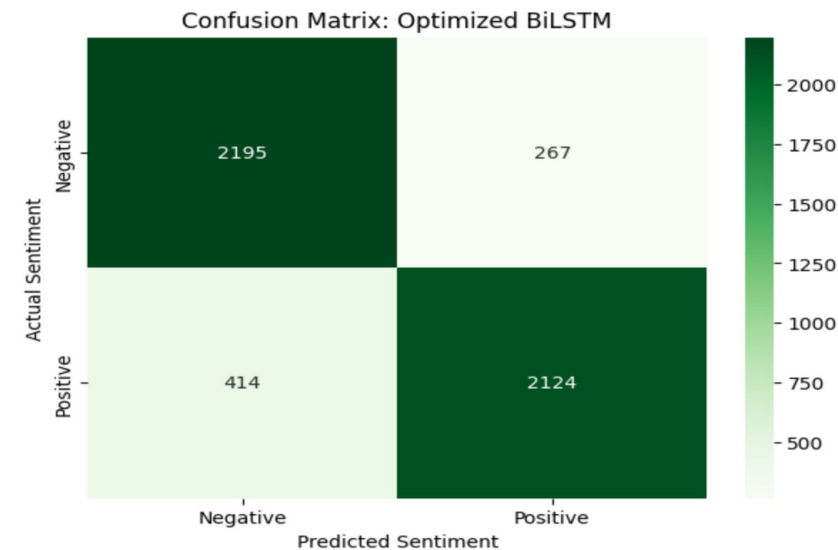
Model Building and Interpretation

BiLSTM (Tuned)

=== Final Evaluation Report: BiLSTM Model===

	precision	recall	f1-score	support
Negative	0.84	0.89	0.87	2462
Positive	0.89	0.84	0.86	2538
accuracy			0.86	5000
macro avg	0.86	0.86	0.86	5000
weighted avg	0.87	0.86	0.86	5000

=== Optimized Best Hyperparameters ===
LSTM Units: 32
Dropout Rate: 0.2
Learning Rate: 0.001



Model Building and Interpretation

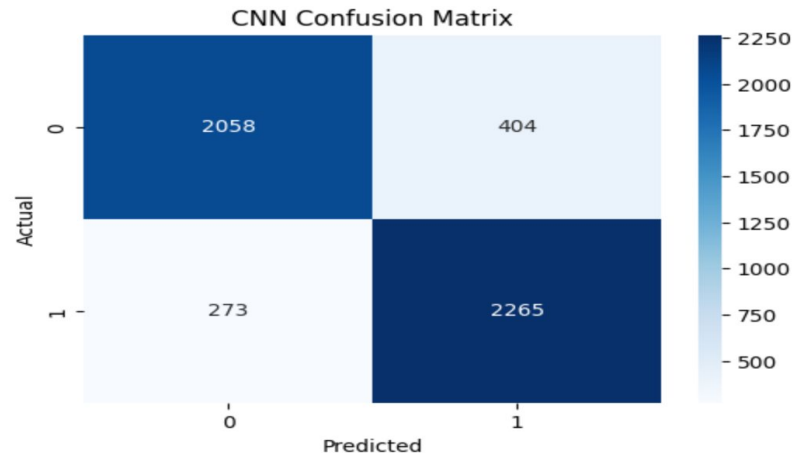
1D CNN (Standard)

```
Epoch 1/10
625/625 ----- 6s 5ms/step - accuracy: 0.6396 - loss: 0.6400 - val_accuracy: 0.8424 - val_loss: 0.3885
Epoch 2/10
625/625 ----- 3s 4ms/step - accuracy: 0.8609 - loss: 0.3629 - val_accuracy: 0.8660 - val_loss: 0.3464
Epoch 3/10
625/625 ----- 2s 4ms/step - accuracy: 0.9234 - loss: 0.2394 - val_accuracy: 0.8668 - val_loss: 0.3736
Epoch 4/10
625/625 ----- 2s 3ms/step - accuracy: 0.9615 - loss: 0.1553 - val_accuracy: 0.8646 - val_loss: 0.4149
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 250, 64)	1,280,000
spatial_dropout1d_2 (SpatialDropout1D)	(None, 250, 64)	0
conv1d_1 (Conv1D)	(None, 246, 64)	20,544
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 64)	0
dense_4 (Dense)	(None, 64)	4,160
dropout_2 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65

Total params: 3,914,309 (14.93 MB)
Trainable params: 1,304,769 (4.98 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2,609,540 (9.95 MB)

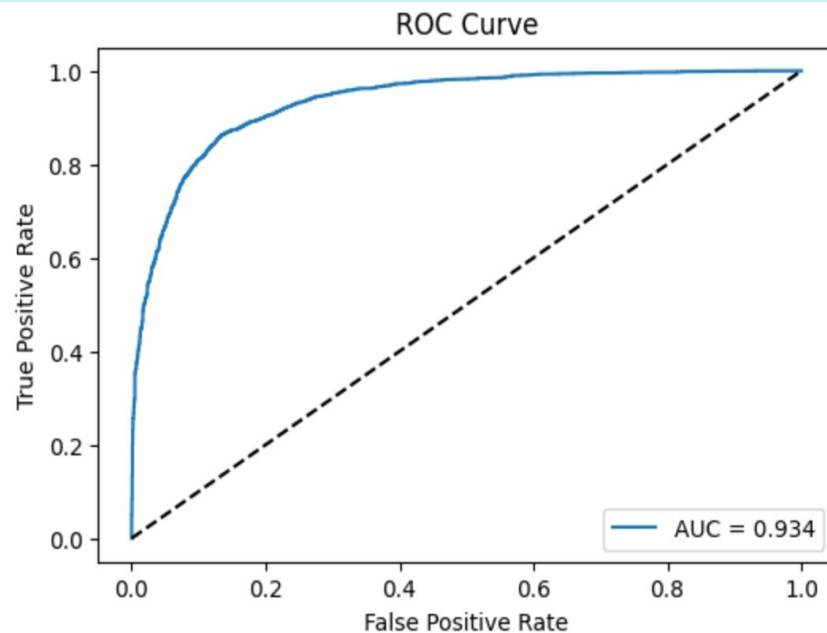
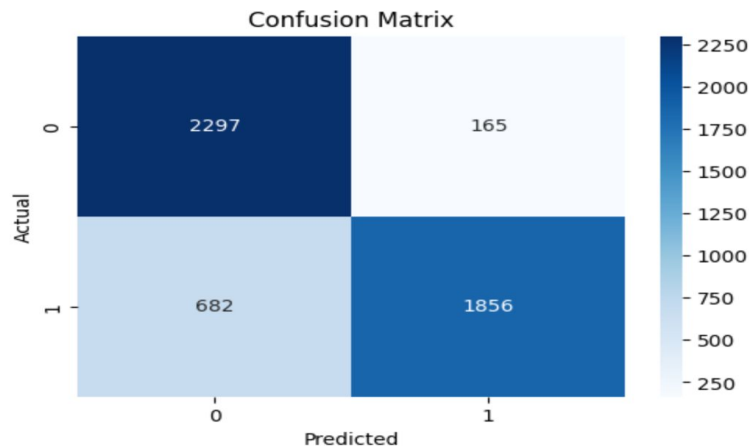
	precision	recall	f1-score	support
0	0.88	0.84	0.86	2462
1	0.85	0.89	0.87	2538
accuracy			0.86	5000
macro avg	0.87	0.86	0.86	5000
weighted avg	0.87	0.86	0.86	5000



Model Building and Interpretation

1D CNN (Tuned)

	precision	recall	f1-score	support
0	0.77	0.93	0.84	2462
1	0.92	0.73	0.81	2538
accuracy			0.83	5000
macro avg	0.84	0.83	0.83	5000
weighted avg	0.85	0.83	0.83	5000



Model Building and Interpretation

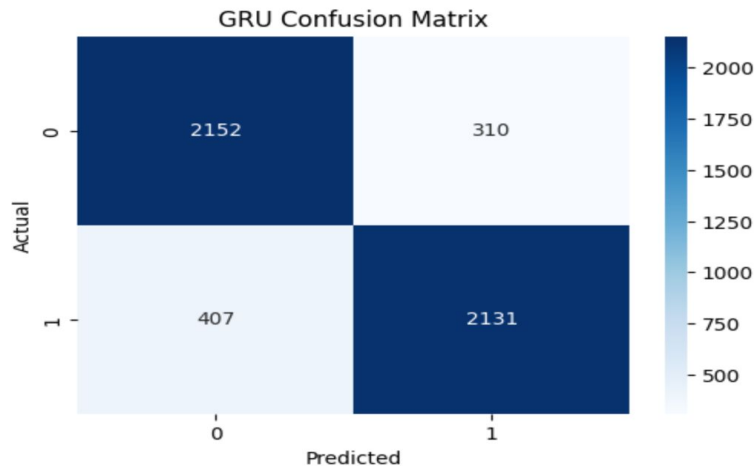
GRU (Standard)

```
... Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input
warnings.warn(
625/625 - 10s - 16ms/step - accuracy: 0.5103 - loss: 0.6931 - val_accuracy: 0.5096 - val_loss: 0.6920
Epoch 2/5
625/625 - 8s - 13ms/step - accuracy: 0.5598 - loss: 0.6690 - val_accuracy: 0.5468 - val_loss: 0.6808
Epoch 3/5
625/625 - 8s - 12ms/step - accuracy: 0.7504 - loss: 0.5023 - val_accuracy: 0.8156 - val_loss: 0.4276
Epoch 4/5
625/625 - 10s - 17ms/step - accuracy: 0.8835 - loss: 0.2938 - val_accuracy: 0.8566 - val_loss: 0.3392
Epoch 5/5
625/625 - 8s - 13ms/step - accuracy: 0.9342 - loss: 0.1804 - val_accuracy: 0.8546 - val_loss: 0.3784
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 250, 64)	1,280,000
gru (GRU)	(None, 64)	24,960
dense_6 (Dense)	(None, 1)	65

Total params: 3,915,077 (14.93 MB)
Trainable params: 1,305,025 (4.98 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2,610,052 (9.96 MB)

	precision	recall	f1-score	support
0	0.84	0.87	0.86	2462
1	0.87	0.84	0.86	2538
accuracy			0.86	5000
macro avg	0.86	0.86	0.86	5000
weighted avg	0.86	0.86	0.86	5000

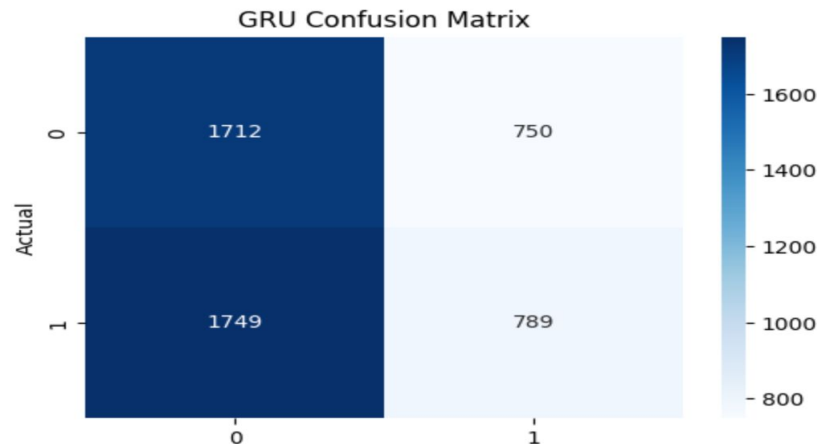


Model Building and Interpretation

GRU (Tuned)

```
{'model_name': 'GRU_tuned', 'best_epoch':  
2, 'best_val_loss': 0.6910777688026428,  
'best_val_accuracy': 0.5153999924659729,  
'units': 32, 'dropout': 0.2, 'dense_units': 64,  
'learning_rate': 0.001}
```

	precision	recall	f1-score	support
0	0.49	0.70	0.58	2462
1	0.51	0.31	0.39	2538
accuracy			0.50	5000
macro avg	0.50	0.50	0.48	5000
weighted avg	0.50	0.50	0.48	5000



Model Summary

Analysis

	Model	Val_Acc	Val_Loss	Precision	Recall	F1
0	Baseline	0.8654	0.3083	0.8905	0.8270	0.8576
1	BiLSTM Standard	0.8562	0.3509	0.8499	0.8704	0.8600
2	BiLSTM Tuned	0.8544	0.3656	0.8441	0.8747	0.8591
3	CNN Standard	0.8660	0.3464	0.8486	0.8924	0.8700
4	CNN Tuned	0.8600	0.3269	0.9184	0.7313	0.8142
5	GRU Standard	0.8566	0.3392	0.8730	0.8396	0.8560
6	GRU Tuned	0.5002	0.6911	0.5127	0.3109	0.3870

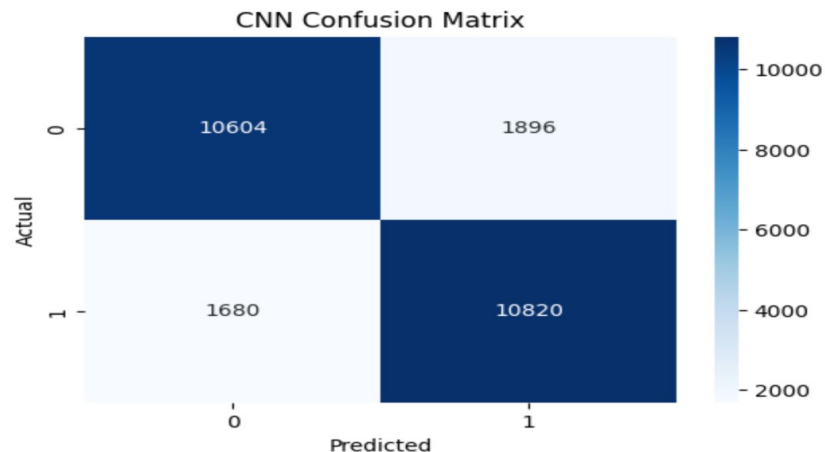
Conclusion

Best Model - CNN Standard

Layer (type)	Output Shape	Param #
embedding_6 (Embedding)	(None, 250, 64)	1,280,000
spatial_dropout1d_5 (SpatialDropout1D)	(None, 250, 64)	0
conv1d_1 (Conv1D)	(None, 246, 64)	20,544
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 64)	0
dense_12 (Dense)	(None, 64)	4,160
dropout_3 (Dropout)	(None, 64)	0
dense_13 (Dense)	(None, 1)	65

Total params: 3,914,309 (14.93 MB)
Trainable params: 1,304,769 (4.98 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2,609,540 (9.95 MB)

	precision	recall	f1-score	support
0	0.86	0.85	0.86	12500
1	0.85	0.87	0.86	12500
accuracy			0.86	25000
macro avg	0.86	0.86	0.86	25000
weighted avg	0.86	0.86	0.86	25000



Things to Improve

- Explore advanced text preprocessing techniques such as lemmatization or stemming instead of relying solely on lowercasing and punctuation removal.
- Retain sentiment-bearing punctuation like “!” and “?” since they can strengthen emotional cues in the text.
- Experiment with additional high-performing architectures, including Transformer-based models such as BERT and other encoder-only models.
- Increase the number of training trials and epochs to allow models more opportunity to converge and improve performance.
- Tune hyperparameters more extensively, including dropout rates, number of filters, and learning rates, to reduce overfitting and enhance generalization.

References

- <https://www.geeksforgeeks.org/nlp/n-gram-in-nlp/>
- <https://www.geeksforgeeks.org/deep-learning/keras-tuner-for-hyperparameter-optimization/>
- <https://www.numberanalytics.com/blog/gru-text-classification>
- https://www.tensorflow.org/api_docs/python/tf/keras/layers/GRU
- https://keras.io/api/layers/convolution_layers/convolution1d/
- TensorFlow Hub. (n.d.). BERT uncased English preprocessing model.
https://tfhub.dev/tensorflow/bert_en_uncased_preprocess/3
- TensorFlow Hub. (n.d.). Small BERT: Uncased L-4 H-512 A-8.
https://tfhub.dev/tensorflow/small_bert/bert_en_uncased_L-4_H-512_A-8/1
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT. <https://arxiv.org/abs/1810.04805>
- TensorFlow: Abadi, M., et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous systems.
<https://www.tensorflow.org/>
- TensorFlow Text. (n.d.). <https://www.tensorflow.org/text>

Challenges

One of the major Challenges I faced in this project was the lack of consistent access to TPU or GPU resources. As a result, the model had to run primarily on CPU, which significantly increased the execution time and slowed down the overall process.

Links

The deliverables for Unsupervised Learning project can be found below:

Video Link :

<https://youtu.be/k-8KcPKCzsY>

GitHub Repository:

<https://github.com/kmadhu181090/MS-AI/tree/main/Pathway%20Machine%20Learning%3A%20Theory%20%26%20Hands-On%20Practice%20with%20Python%20Specialization/CSCA%205642%3A%20Introduction%20to%20Deep%20Learning>

Thank You

